Retour TP 2

ITC - 2025/2026

Félix POTIÉ

Fonctions

les () et donc les arguments sont collés (sans espace) aux noms de fonctions que ca soit pour une fonction native de python ou une fonction que vous avez créée vous.

```
print(x)
int(4.6)
range(10)
moyenne([1,5,2,6])
# et non pas
print (x)
int (4.6)
range (10)
moyenne ([1,5,2,6])
```

Nom des variables snake case:

- en minuscule
- si plusieurs mot séparé d'un _
- nom clair !!
- en général pour les indices d'une liste on utilise `i` (n'utilisez pas `i` pour un élément d'une liste)
- ne pas utiliser le même nom que le nom d'une fonction
- ne pas utiliser 2 fois le même nom de variable dans une même fonction

Boucle infini :

Attention, si vous faites des boucles `while` il faut vérifier que la condition d'arrêt va être atteinte. Sinon vous allez faire une boucle infini et le site va *planter*. Pensez à sauvegarder souvent votre travail au cas où ça vous arrive.

S'il vous plait si vous avez une boucle infini, corriger là, si vous n'y arrivez pas commenter votre code que je ne fasse pas planter mon site quand je corrige vos TP.

return

`return` est un mot clé comme `if`, `for`, `while`, `def`, ... et non une fonction comme `print()`, `int()`, `range()`, ... Donc on ne met pas de `()` autour de ce qu'on veut retourner.

```
return(x)
#vs
return x
```

Utilité des tests

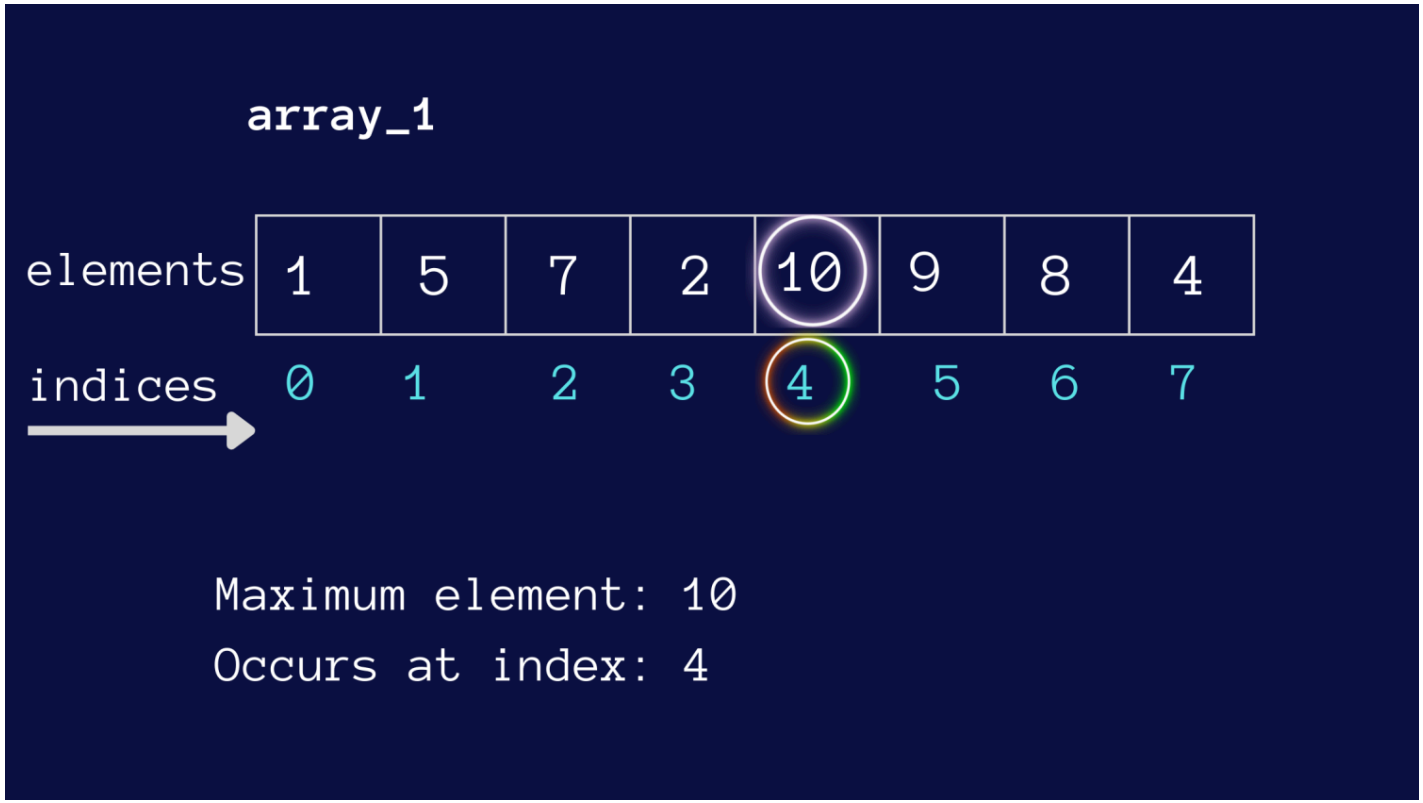
```
def moyenne(notes):
    return sum(notes) / len(notes)

print(moyenne([12, 15, 14]))  # ok
print(moyenne([])) # erreur de division par 0 !
```

En général on test ce qu'on appelle les cas de bord (ici la liste vide) pour vérifier que notre fonction marche dans tous les cas.

Itérer sur une liste

Itérer signifie passer en revue tous les éléments d'une liste, on peut aussi parcourir une liste. Comme vu dans le TP il y a plusieurs façons de le faire.



Itérer sur une liste - 2

Itérer sur les éléments d'une liste :

```
liste = [3, 5, 2, 8]
for element in liste:
    print(element)
```

Ici on n'a pas accès à l'indice de l'élément. C'est en général la méthode la plus utilisée car la plus simple.

Cependant parfois on a besoin de l'indice, il faut donc utiliser une autre méthode.

Itérer sur une liste - 3

Itérer sur les indices d'une liste :

```
for i in range(len(liste)):
    print(liste[i])
```

```
i = 0
while i < len(liste):
    print(liste[i])
    i += 1
```

Fonctions

But réutiliser du code, éviter les répétitions, facilité les algorithmes plus complexes.

Question 4

En utilisant les fonctions précédentes, écrivez une fonction `est_parfait(n)` qui renvoie `True` si l'entier `n` est

```
def est_parfait(n):
    return somme_1(liste_diviseurs(n))==n
```

```
def est_parfait(n):
    s=0
    for i in range(1,n):
        if n%i==0:
            s=s+i
    return s==n #si la somme des diviseurs donnent n alors le booléen affichera true sino

```

Booléens et conditions

Attention à ne pas écrire du code inutile.

```
if est_parfait(i) == True:
    nb_parfait.append(i)

#vs

if est_parfait(i):
    nb_parfait.append(i)
```

Booléens et conditions - 2

Attention à ne pas écrire du code inutile.

```
if n == somme(liste_diviseurs(n)):
    return True
else:
    return False

#vs
return n == somme(liste_diviseurs(n))
```

Retour exercice 2 et 3

Fonctionnement général

1. TP en cours
2. Finir à la maison pour le dimanche soir si besoin
3. Je corrige 1/3 des TP
4. Je donne des retours généraux en début de cours
5. Je met la correction du TP sur cahier de prépa
6. Vous regardez les retours et corrigez vos TP en autonomie à la maison
7. Vous pouvez me poser des questions par mail ou en fin de cours si besoin

Comme ça on repart tous du même endroit en TP et on a pas trop d'écart en fonction de l'avancement de chacun.

Liste vide

```
if liste == []:
    ...
# vs
if len(liste) == 0 :
    ...
```

while

```
while i <= len(l)-1 :
    i += 1
#vs
while i < len(l) :
    i += 1
```

Nombre de ‘a’

Ex 2 : Q4

Déterminer le nombre de “a” et de “e” dans l’extrait suivant.

Il suffit d’utiliser la fonction `liste_positions()` et de regarder la longueur de la liste retournée.

Comparaison à 3 éléménts

```
# il ne faut pas faire
if max1 > i > max2 :
    ...
# il faut faire
if max1 > i and i > max2 :
    ...
```

Parametre de fonction

À ne surtout pas faire !!

```
def appartient(x, t):
    t = []
    x = 0
```

Ici on écrase les paramètres d'entrée, donc on ne peut plus utiliser la fonction correctement.

for et range

Je ne veux plus voire ça !!

```
i =  0
for i in range(len(liste)):
    # faire qq chose
    i += 1
```

Ici i prends les valeurs de range() qui va de 0 à len(liste)-1 de un en un.Donc le i = 0 et le i+=1 sont déjà fait par le range !

```
for i in range(len(liste)):
    # faire qq chose
```

Par contre avec le while, on ne peut pas utiliser range donc il faut!

Parcourt caché

Question 2

Ecrire une fonction `min_et_max(liste)` qui **affiche** le minimum et le maximum d'une liste de nombres. Vous ne ferez cette liste. Pensez à utiliser un `assert` pour vérifier que la liste contient au moins 2 éléments.

```python
# Code à compléter
def min_et_max(liste):
    assert len(liste) >= 2
    min = liste[0]
    for terme in liste:
        if terme < min:
            min = terme
    return str(min) + " est le minimum" + " et " + str(maximum(liste)) + " est le maximum"
```

Ici on parcours 2 fois la liste car `maximum()` parcourt la liste.

Il faut faire attention à ce genre de détail. On reverra celà avec les compléxités en algorithmique (qui calcule la performance d'un algorithme).

Parcourt caché - 2

Question 5

Ecrire une fonction `indice_second_max(liste)` qui renvoie une des positions du second maximum en ne parcou
fois.

```
# Code à compléter
def indice_second_max(liste):
    for i in range (len(liste)):
        if second_max(liste)==liste[i]:
            return i
```

Ici par exemple on parcours la liste len(liste) fois, alors qu'on pourrait le faire en une seule fois.

Par exemple pour une liste de taille 1000, on fait 1000*1000 = 1 000 000 d'*opérations* au lieu de 1000.

2eme max

Corrigeons cette algorithme ensemble.

```
def second_max(liste):
    max_1 = 0
    max_2 = 0
    for el in liste:
        if max_1 < el:
            max_1 = el
        elif max_2 < el:
            max_2 = el
    return max_2
```

Problème 1 : Initialisation de max_1 et max_2

Problème 2 : second_max([4,1,5]) -> 1

liste.remove(x)

`liste.remove(x)`

`liste.remove(x)` : supprime la première occurence de x dans liste.
Attention si x n'est pas dans liste `liste.remove(x)` renvoie une erreur.
Exemple :

```
t = [1,3,4,3]
t.remove(3)
print(t) # [1,4,3]
```

assert

L'objectif principal des assertions est de détecter les bugs rapidement.

On s'en sert principalement pour vérifier qu'une fonction est appelée avec des arguments valides.

```
def moyenne(notes):
    assert len(notes)>0,"notes ne doit pas être vide"
    return sum(notes) / len(notes)
```